

Arikina B2B Stablecoin Escrow

Use Case Reference — Stellar Testnet

This document describes the five escrow scenarios built on the Stellar testnet as part of Arikina's B2B ingredient payment infrastructure. Each scenario maps to a real trade finance use case — from clean delivery to disputed shipments — and demonstrates how programmable escrow can replace traditional letters of credit and wire-based payment terms at a fraction of the cost.

Architecture Overview

Network	Stellar Testnet (Horizon API)
Escrow model	Per-PO account with 2-of-2 multisig (Buyer + Oracle)
Signers	Buyer (weight 1) · Oracle (weight 1)
Low threshold	1 — Oracle alone can write data entries (shipment confirmation, verdict)
Med threshold	2 — Buyer + Oracle must co-sign all payment operations
High threshold	2 — Buyer + Oracle must co-sign account option changes
On-chain data	PO number · ingredient · state · deadline · tracking ID · dispute metadata
Settlement	~3–5 seconds · \$0.017 fee per transaction vs \$96 traditional wire
CLI	<code>python -X utf8 escrow.py [happy deadline cancel quality nondelivery]</code>

Scenario 1 — Full Delivery (Happy Path)

```
python -X utf8 escrow.py happy
```

Business Context

Standard PO fulfillment. Buyer locks payment in escrow before shipment. Supplier ships goods, oracle records tracking ID on-chain. On confirmed delivery, buyer and oracle co-sign release — supplier receives full payment in seconds, not days.



Entry point	Buyer has funded wallet. Supplier agrees to PO terms.
Step 1	Buyer calls <code>create_escrow()</code> — new Stellar account created, 2-of-2 multisig set, PO number + ingredient + delivery deadline written on-chain.
Step 2	Buyer calls <code>fund_escrow()</code> — deposits 1,700 XLM (~\$1,700). Supplier can verify payment is locked at escrow address before preparing shipment.

Step 3	confirm_shipment(tracking_id) — oracle writes tracking ID + shipped_at timestamp. Buyer co-signs (med threshold = 2). Data is immutable.
Step 4	confirm_delivery() — buyer + oracle co-sign. Full spendable balance sent to supplier atomically. Settlement completes in ~3s.
Exit outcome	State = RELEASED. Supplier balance increases by payment amount. Escrow holds only minimum reserve (~5 XLM).
Wire equiv.	\$96 fee (wire) → \$0.017 (Stellar). 3–5 days → 3 seconds.
Exit Outcome: State → RELEASED Supplier receives 100% of PO amount ~3s settlement	

Scenario 2 — Delivery Deadline Expired

python -X utf8 escrow.py deadline

Business Context

Supplier takes payment (escrow funded) but fails to deliver within the agreed window — either ghosting on the order or experiencing a fulfillment failure. The delivery deadline is written on-chain at escrow creation. When it passes, the oracle — a programmatic service monitoring the ledger — automatically co-signs a full refund. No human arbitration required.



Entry point	State = FUNDED. funded_at + DELIVERY_WINDOW_SECS has passed with no confirm_delivery.
Trigger	Oracle daemon polls Horizon each block, reads deadline data entry, compares to current ledger close time. When expired, oracle auto-signs refund.
Condition	time.time() – funded_at > DELIVERY_WINDOW_SECS (demo: 30 seconds / production: 30 days)
On-chain record	state=REFUNDED · refund_reason=DEADLINE_EXPIRED · refunded_at=
Signatures	Buyer + Oracle (oracle programmatically obligated to sign on deadline breach — this is the 'smart contract' equivalent of a time-lock clause)
Guard	Refund is blocked if deliver_deadline has not yet passed. Script waits and notifies remaining time.
Exit outcome	State = REFUNDED. Full balance returned to buyer. Supplier has no claim.

Exit Outcome: State → REFUNDED | Buyer receives 100% refund | Deadline breach recorded on-chain

Scenario 3 — Pre-Shipment Cancellation

`python -X utf8 escrow.py cancel`

Business Context

Either party mutually agrees to cancel the PO before goods are dispatched. Common in ingredient sourcing when crop yields change, quality certifications are not met, or buyer specifications are revised post-order. The smart contract enforces a hard guard: if a tracking ID is already on-chain, cancellation is rejected — the dispute path must be used instead.



Entry point	State = FUNDED. No tracking ID recorded. Both parties agree to cancel.
Hard guard	Raises ValueError if ctx.tracking_id is set (goods already in transit). This is enforced in code — not just a business rule.
On-chain record	state=CANCELLED · cancelled_at=
Signatures	Buyer + Oracle (mutual consent — oracle confirms no shipment was recorded)
Distinction	Cancel vs Refund: Cancel = pre-shipment mutual agreement (clean). Refund = deadline breach (supplier fault). Both end with buyer recovery but carry different on-chain reason codes for audit/insurance.
Exit outcome	State = CANCELLED. Full balance returned to buyer immediately. No penalty, no dispute, no waiting.

Exit Outcome: State → CANCELLED | Buyer receives 100% refund | Mutual consent, no penalty

Scenario 4 — Quality Dispute (Partial Settlement)

`python -X utf8 escrow.py quality`

Business Context

Goods arrive but fail to meet specification — wrong grade, short weight, contamination, or improper certification. The buyer raises a dispute with documented evidence. The oracle (acting as trade finance arbitrator) reviews the claim and rules on a split: the fraction of the PO that was legitimately fulfilled goes to the supplier; the remainder is refunded to the buyer. Both transfers happen atomically in a single Stellar transaction — something no wire transfer system can do.



Entry point	State = SHIPPED. Buyer calls dispute_quality(reason, supplier_pct).
-------------	---

supplier_pct	Fraction of escrow to release to supplier (0.0–1.0). Example: 0.70 → supplier gets 70%, buyer gets 30% back.
Phase 1	Oracle writes dispute_reason + supplier_pct + state=DISPUTED on-chain (low threshold = 1, oracle alone can write data). Buyer co-signs.
Phase 2	Oracle + buyer co-sign atomic settlement transaction containing two payment ops: supplier_amount + buyer_amount. Both execute or neither does.
Example	PO: 200kg Shea Butter at \$1,700. Delivered: 140kg (30% short). Oracle rules 70/30. Supplier receives \$1,190. Buyer receives \$510.
On-chain record	state=DISPUTE_RESOLVED · dispute_reason · supplier_pct · resolved_at
Key insight	Atomic split settlement is impossible with wire transfers. Stellar executes both transfers in one ledger close (~3s).
Exit outcome	State = DISPUTE_RESOLVED. Funds split per ruling. Both parties settle simultaneously.

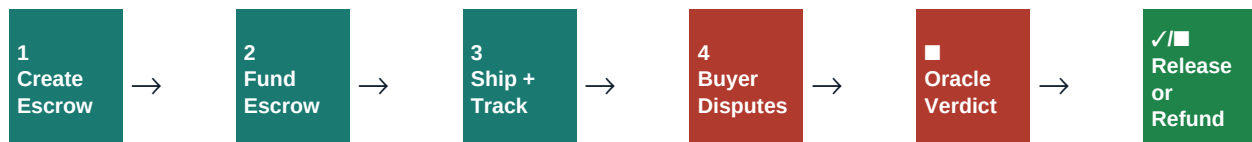
Exit Outcome: State → DISPUTE_RESOLVED | Supplier receives 70% | Buyer receives 30% | Atomic split in ~5s

Scenario 5 — Non-Delivery Dispute (Oracle Verdict)

`python -X utf8 escrow.py nondelivery`

Business Context

Carrier tracking shows the shipment as delivered, but the buyer claims the goods were never received — a common scenario involving porch theft, wrong delivery address, or fraudulent tracking updates. The oracle investigates (carrier GPS data, proof of delivery photos, signed receipt) and issues a binding verdict. The verdict is written on-chain before settlement, creating an immutable audit trail usable for insurance claims or legal proceedings.



Entry point	State = SHIPPED. Carrier shows delivered. Buyer calls dispute_nondelivery(verdict).
oracle_verdict	BUYER_WINS → full refund to buyer (carrier/supplier liable) SUPPLIER_WINS → full release to supplier (buyer claim rejected)
Phase 1	Oracle writes oracle_verdict + dispute_reason=NON_DELIVERY + state=DISPUTED on-chain. Low threshold (1) — oracle can do this unilaterally. Buyer co-signs.
Phase 2	Oracle + buyer co-sign payment to the verdict-winning party. Verdict is immutably recorded before funds move.
Audit trail	Verdict tx hash provides court-admissible timestamp of oracle ruling. All evidence reasoning can be appended as additional ManageData entries.
Oracle role	Mirrors a letter of credit issuing bank — independent arbitrator who holds no funds but controls their release. Fully automated, on-chain.

Exit outcome	State = REFUNDED (buyer wins) or RELEASED (supplier wins). Full amount sent to ruling party.
--------------	--

Exit Outcome: State → REFUNDED or RELEASED | Oracle verdict immutably recorded | Full settlement in ~4s

Scenario Comparison

Scenario	Entry State	Exit State	Signers	Settlement
1 · Happy Path	FUNDED → SHIPPED	RELEASED	Buyer + Oracle	~3s · \$0.017
2 · Deadline Expiry	FUNDED (no delivery)	REFUNDED	Buyer + Oracle (oracle auto)	~3s after deadline
3 · Cancel	FUNDED (no tracking)	CANCELLED	Buyer + Oracle	~3s
4 · Quality Dispute	SHIPPED	DISPUTE_RESOLVED	Buyer + Oracle	~5s · atomic split
5 · Non-Delivery	SHIPPED	REFUNDED/RELEASED	Buyer + Oracle (oracle verdict)	~4s

Payment Economics — \$1,700 PO Example

Method	Fixed Fee	FX / Variable	Total Cost	Settlement Time
Traditional Wire	\$45.00	3.0% (\$51.00)	\$96.00	3–5 business days
Letter of Credit	\$150–\$500	0.5–2.5%	\$200–\$1,000+	5–10 days
Stellar Escrow	\$0.00001	0.001%	\$0.017	3–5 seconds

99.98% cost reduction vs traditional wire. For Arikina's typical ingredient order volume (~\$2M/year across 40+ POs), Stellar settlement eliminates ~\$38,400/year in wire fees and FX markups.

Chris Mahadeo · chris@arikina.com · Arikina Ingredients & Supply Co. · Stellar Testnet Demo — April 2026

Due to ongoing market volatility, pricing and availability remain subject to change without notice. XLM used as USD proxy on testnet. Production deployment would use Circle USDC on Stellar.